

Multi-Tenant Agent Platform

AWS Architecture Design Document

Document type	Architecture Design + Implementation Guide
Version	2.0 (Research-validated, May 2026)
Frameworks	AWS AgentCore · Strands Agents 1.0 · CopilotKit / AG-UI · IBM ContextForge
Auth stack	Microsoft Entra ID · Custom JWT Service · Lambda Authorizer · AWS IAM
Status	Approved for Implementation

This document provides the complete end-to-end design of a production-grade, multi-tenant AI agent platform built on AWS. It covers the full vertical stack from the React user interface (CopilotKit + AG-UI protocol) through the BFF and API Gateway authentication tiers, into the AgentCore Runtime hosting Strands-based skill agents, through MCP tool integration (IBM ContextForge + AgentCore Gateway), to cross-tenant A2A agent invocation — with authentication and fine-grained tenancy (FT) rights enforced at every layer.

Table of Contents

1. Architecture Overview

- 1.1 Technology Stack (Research-Validated)
- 1.2 Layered Architecture Model
- 1.3 Multi-Tenant Isolation Strategy

2. UI Layer — CopilotKit + AG-UI

- 2.1 AG-UI Protocol — 16 Event Types
- 2.2 Option A: HttpAgent → AgentCore Runtime
- 2.3 Option B: MCPAppsMiddleware → AgentCore Gateway
- 2.4 Option Comparison and Hybrid Approach
- 2.5 CopilotRuntime Authentication Middleware

3. BFF Layer — Human and Machine Channels

- 3.1 Human BFF (ECS Fargate + WebSocket)
- 3.2 Machine BFF (Lambda + Async SQS)
- 3.3 Token Exchange Flow

4. Auth & Rights — Full Design

- 4.1 The Rights Fingerprint Pattern
- 4.2 Custom JWT Token Service
- 4.3 Lambda Authorizer Implementation
- 4.4 ElastiCache Rights Hydration
- 4.5 Rights Change Invalidation

5. AgentCore Runtime + Strands Skills

- 5.1 AgentCore Components (GA Features)
- 5.2 Strands Orchestrator Agent
- 5.3 Skill-Based Architecture
- 5.4 Session Management
- 5.5 AgentCore Identity (Inbound + Outbound)

6. MCP + Tool Layer

- 6.1 AgentCore Gateway
- 6.2 IBM ContextForge MCP Federation
- 6.3 IBM API Connect Integration
- 6.4 MCP Proxy for AWS (SigV4)

7. Cross-Tenant A2A Design

- 7.1 Cross-Tenant Token Propagation
- 7.2 PrivateLink Routing
- 7.3 Cross-Tenant Policy Table

8. Observability + Operations

- 8.1 AgentCore Observability (CloudWatch)
- 8.2 OTEL Instrumentation
- 8.3 Audit Trail Design

9. CDK Infrastructure Patterns

10. Anti-Patterns & Best Practices

11. Implementation Roadmap

1 Architecture Overview

Multi-tenant agent platform — AWS

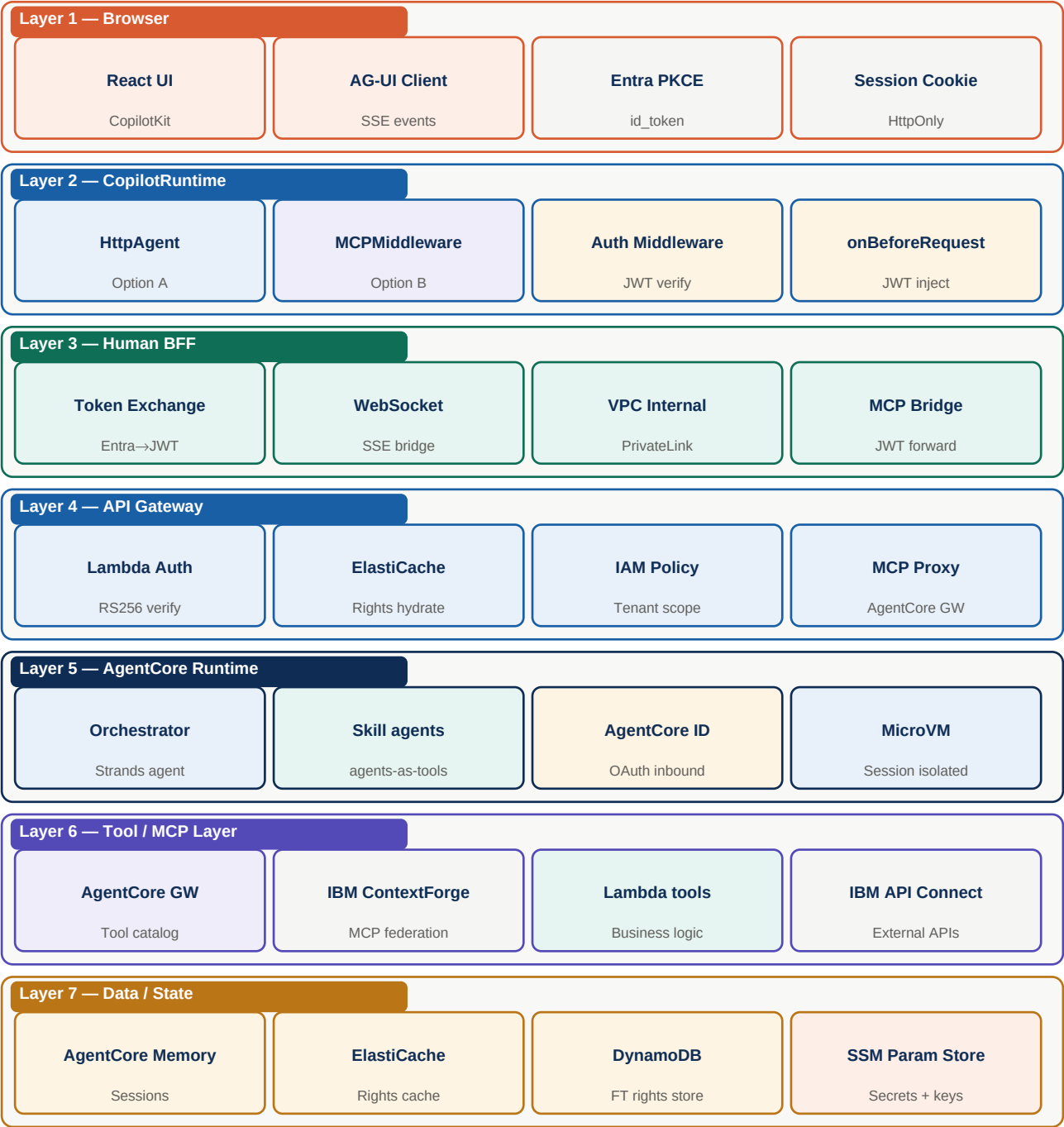
1.1 Technology Stack (Research-Validated)

The following components have been selected based on production capabilities as of May 2026. All referenced services are Generally Available (GA) unless noted.

Component	Technology	Role	Status
Agent Runtime	Amazon Bedrock AgentCore Runtime	Serverless microVM execution, 8hr sessions, A2A support	GA Oct 2025
Agent Framework	AWS Strands Agents 1.0	Model-driven agents, skills-as-tools, SessionManager, async	GA Sep 2025
Tool Hub	AgentCore Gateway	MCP server, Lambda/REST→MCP, semantic tool search, IAM auth	GA Oct 2025
Memory	AgentCore Memory	Session + long-term memory, AgentCoreMemorySessionManager	GA Oct 2025
Identity	AgentCore Identity	OAuth 2.0, workload tokens, vault, delegated user access	GA Oct 2025
Observability	AgentCore Observability + CloudWatch	OTEL-compatible, Datadog/Dynatrace/LangSmith integration	GA Oct 2025
MCP Proxy	MCP Proxy for AWS	SigV4-signed MCP client for cross-account/cross-tenant calls	GA 2025
UI Framework	CopilotKit 1.51 + AG-UI Protocol	React/Next.js agent UI, 16 event types, generative UI, HITL	GA Jan 2026
External Tools	IBM ContextForge (open source)	MCP federation gateway, A2A agent registration, rate limiting	GA 2025
External API Mgmt	IBM API Connect	Enterprise API lifecycle, MCP server generation, DataPower	GA
User Identity	Microsoft Entra ID	OIDC/OAuth 2.0 IdP, PKCE for humans, client-cred for machines	GA
Token Service	Custom JWT Service (Lambda)	Entra→internal JWT exchange, FT rights fingerprint, SSM keys	Custom build
Rights Cache	Amazon ElastiCache (Redis)	Sub-millisecond FT rights hydration, 15-min TTL	GA
API Gateway	AWS API Gateway + Lambda Authorizer	JWT validation, rights hydration, IAM policy emission	GA

1.2 Layered Architecture Model

The platform is structured as seven distinct layers, each with its own auth boundary, isolation scope, and responsibility. Traffic flows top-down; auth tokens flow with every hop and are independently verified at layers 2, 4, 5, and 6.



1.3 Multi-Tenant Isolation Strategy

Tenant isolation is enforced at the infrastructure, identity, data, and code levels. The following table summarises the isolation boundary at each layer:

Layer	Isolation mechanism	Scope
AgentCore Runtime	One deployment per BU tenant; recommended separate AWS account	Hard
AgentCore MicroVM	Isolated CPU/memory/filesystem per user session; no shared memory	Hard
JWT / Rights	tenant_id claim in every token; rights scoped to that tenant only	Cryptographic
DynamoDB	LeadingKey condition on all queries = tenant_id; no cross-tenant scan possible	IAM + SDK
S3 Sessions	Bucket per tenant (sessions-{tenant_id}); cross-bucket access denied	IAM
SSM Parameters	Path /tenants/{tenant_id}/* — IAM policy restricts per-tenant read	IAM
AgentCore Memory	AgentCoreMemorySessionManager scoped to tenant_id + session_id	SDK
ElastiCache	Key prefix rights:{tenant_id}:{fp} — no cross-tenant key access	Key design
Skill agents	@requires_right validates from JWT context (not payload); tenant from JWT claims	Code

2 UI Layer — CopilotKit + AG-UI Protocol

React / Next.js · @copilotkit/react-ui v1.51 · AG-UI protocol

2.1 AG-UI Protocol

AG-UI is an open, event-based protocol (created by CopilotKit, adopted by AWS, Google ADK, Microsoft Agent Framework, Oracle, LangChain, and others) that defines how AI agent backends communicate with user-facing applications in real time. It provides 16 standardised event types transmitted over SSE or WebSocket, enabling bi-directional state sync, streaming, human-in-the-loop approvals, and generative UI rendering.

AG-UI 16 event types:

Category	Events	Purpose
Lifecycle	RUN_STARTED · RUN_FINISHED · RUN_ERROR	Marks start/end/failure of an agent execution run
Text streaming	TEXT_MESSAGE_START · TEXT_MESSAGE_CONTENT · TEXT_MESSAGE_END	Streams agent text tokens to UI in real time
Tool calls	TOOL_CALL_START · TOOL_CALL_ARGS · TOOL_CALL_END	Shows tool invocations to user with parameters and results
State sync	STATE_SNAPSHOT · STATE_DELTA · MESSAGES_SNAPSHOT	Keeps agent internal state and conversation in sync with UI
Steps	STEP_STARTED · STEP_FINISHED	Tracks multi-step agent reasoning (each ReAct iteration)
Human loop	HUMAN_TURN_START	Agent pauses and asks user for input, approval, or confirmation
Generative UI	RENDER_UI	Agent pushes a React component spec to render inline in chat

2.2 Option A — HttpAgent to AgentCore Runtime (Full AG-UI)

The HttpAgent connects the CopilotRuntime to a full AG-UI-compatible FastAPI endpoint running inside AgentCore Runtime. The Strands orchestrator runs on AgentCore; CopilotKit is a pure transport and UI layer. This is the recommended option for complex, stateful, multi-turn agent interactions where humans need to see agent state, approve actions, or receive dynamically rendered UI components.

- **State sync:** Full bi-directional via STATE_SNAPSHOT / STATE_DELTA — useCoAgent() hook keeps React state in sync
- **Human-in-the-loop:** Native — agent emits HUMAN_TURN_START; CopilotKit renders renderAndWait() component
- **Generative UI:** Agent emits RENDER_UI with component spec; CopilotKit renders React component inline
- **Thread persistence:** CopilotKit threadId = AgentCore sessionId — same microVM session reused
- **Auth:** onBeforeRequest hook injects Authorization + X-Session-Id + X-Tenant-Id on every POST

CopilotRuntime setup — Option A:

```
// app/api/copilotkit/route.ts
import { CopilotRuntime, ExperimentalEmptyAdapter,
copilotRuntimeNextJSAppRouterEndpoint } from "@copilotkit/runtime";
import { HttpAgent } from "@ag-ui/client";
```

```

export const POST = async (req: NextRequest) => {
  const internalJWT = await extractAndValidateJWT(req); // BFF token exchange
  const tenantId = extractTenantId(internalJWT);

  const runtime = new CopilotRuntime({
    agents: {
      orchestrator: new HttpAgent({
        url: process.env.AGENTCORE_BFF_ENDPOINT!,
        onBeforeRequest: ({ ctx }) => ({
          headers: {
            "Authorization": `Bearer ${internalJWT}`,
            "X-Tenant-Id": tenantId,
            "X-Session-Id": ctx.threadId ?? crypto.randomUUID(),
            "Mcp-Session-Id": ctx.threadId,
          },
        }),
      },
    },
    middleware: {
      onBeforeRequest: async ({ messages }) => {
        await auditLog({ tenantId, messages });
        return { messages };
      },
    },
  });

  const { handleRequest } = copilotRuntimeNextJSAppRouterEndpoint({
    runtime, serviceAdapter: new ExperimentalEmptyAdapter(),
    endpoint: "/api/copilotkit",
  });

  return handleRequest(req);
};

```

2.3 Option B — MCPAppsMiddleware to AgentCore Gateway

MCPAppsMiddleware lets CopilotKit's built-in BuiltInAgent (which runs the LLM loop inside the CopilotRuntime process, not on AgentCore) discover and call tools exposed by AgentCore Gateway's MCP endpoint. This is ideal for lightweight inline assistants, form helpers, or C/D automation callers that only need tool access without full state sync.

```

// Option B: MCPAppsMiddleware
import { BuiltInAgent } from "@copilotkit/runtime/v2";
import { MCPAppsMiddleware } from "@ag-ui/mcp-apps-middleware";

const agent = new BuiltInAgent({
  model: "anthropic/claude-sonnet-4-5",

```

```
prompt: `You are an assistant for tenant ${tenantId}.`,
}).use(new MCPAppsMiddleware({
mcpServers: [
{
type: "http",
url: `${process.env.AGENTCORE_GATEWAY_URL}/tenants/${tenantId}/mcp`,
serverId: `gateway-${tenantId}`,
headers: { "Authorization": `Bearer ${internalJWT}`, "X-Tenant-Id": tenantId },
},
{
type: "http",
url: `${process.env.CONTEXTFORGE_URL}/rpc`,
serverId: "contextforge-external",
headers: { "Authorization": `Bearer ${internalJWT}` },
},
],
// Filter tools based on user rights – prevents listing unauthorised tools
toolFilter: (tool) => {
const req = tool.annotations?.required_right as string;
return !req || userRights.includes(req);
},
}));
```

2.4 Option Comparison

Dimension	Option A — HttpAgent	Option B — MCPAppsMiddleware
LLM loop location	AgentCore Runtime (Strands)	CopilotRuntime BuiltInAgent
State sync	Full bi-directional (STATE_SNAPSHOT/DELTA)	Tool results only
Human-in-the-loop	Native (HUMAN_TURN_START event)	Custom tool approval flow
Generative UI	Native (RENDER_UI event)	MCP App spec rendering
Thread persistence	AgentCore SessionManager → S3	CopilotRuntime in-memory
Auth attachment	onBeforeRequest per POST	Static headers per MCP server
Ideal for	Rich dashboard, complex workflows, B users	Inline helpers, form assistants, C/D callers
Recommendation	Primary channel for human users	Secondary / embedded use cases

3 BFF Layer — Human and Machine Channels

ECS Fargate (human) · Lambda (machine) · VPC internal

Two distinct Backend-for-Frontend services handle the different channel types. Mixing them is an anti-pattern: human BFF needs WebSocket/SSE streaming and session management; machine BFF needs stateless, high-throughput async invocation. Both funnel through the same API Gateway and Lambda Authorizer.

Concern	Human BFF (ECS Fargate)	Machine BFF (Lambda)
Auth flow	PKCE → Entra id_token → internal JWT	Client credentials → Entra access_token → internal JWT
Session	HttpOnly session cookie, server-side session store	Stateless — no session; correlationId per call
Response mode	SSE / WebSocket streaming (real-time tokens)	POST → 202 jobId; GET /results/{jobId} (async polling / callback)
Token cache	In-memory LRU keyed by session; TTL = JWT exp - 60s	Lambda memory cache; cold start re-exchanges token
VPC routing	SG: only outbound to APIGW; no inbound from internet	Lambda in VPC; same SG rules
Scale	ECS auto-scaling on CPU/memory; Fargate Spot for cost	Lambda concurrency reservation per tenant

4 Auth & Rights — Full Design

Entra ID · Custom JWT Service · Lambda Authorizer · ElastiCache

4.1 The Core Problem: Rights Size vs Lambda Authorizer Limits

The Lambda Authorizer returns a context object with an 8 KB hard limit. A typical enterprise FT rights set (50–200 resource permission tuples) serialised to JSON exceeds this. The solution is the **rights fingerprint pattern**: the token carries a deterministic hash of the rights set; the authorizer hydrates the actual rights from ElastiCache in <2 ms.

4.2 Rights Fingerprint Flow

Step	Component	Action	Latency
1	Browser → BFF	Send Entra id_token in POST body	0 ms
2	BFF → JWT Token Svc	Validate Entra token (JWKS cache); look up FT rights from DynamoDB	10–20 ms
3	JWT Token Svc	Compute fingerprint = SHA256(sort(rights[])) + tenant_id + user_id	0 ms
4	JWT Token Svc → ElastiCache	SETEX rights:{tenant_id}:{fp} → gzip(rights_json), TTL=900s	1 ms
5	JWT Token Svc → BFF	Return JWT: {sub, tenant_id, rights_fp, exp:+900s}. No rights in token.	0 ms
6	APIGW → Lambda Authorizer	Verify RS256 signature; extract claims	0 ms
7	λ Authorizer → ElastiCache	GET rights:{tenant_id}:{fp} → hydrate full rights	1–2 ms
8	λ Authorizer → APIGW	Return IAM Allow policy + context {tenant_id, user_id, rights_json (route-scoped)}	0 ms
9	APIGW → AgentCore	Forward request with context headers; AgentCore Identity validates OAuth Bearer	0 ms

4.3 JWT Token Service — Key Implementation Details

```
# jwt_token_service/handler.py (Lambda, Python)

import hashlib, json, time, boto3
from jose import jwt

RIGHTS_TABLE = boto3.resource("dynamodb").Table("ft-rights")

def handler(event, context):
    claims = validate_entra_token(event["entraToken"]) # JWKS verify
    user_id = claims["oid"]
    tenant_id = claims["tid"]

    rights = RIGHTS_TABLE.get_item(
        Key={"pk": f"rights#{tenant_id}", "sk": f"user#{user_id}"}
```

```

).get("Item", {}).get("rights", [])

rights_sorted = sorted(rights, key=lambda r: r["resource"])
fingerprint = hashlib.sha256(
    f"{tenant_id}:{user_id}:{json.dumps(rights_sorted, sort_keys=True)}".encode()
).hexdigest()

redis_client.setex(f"rights:{tenant_id}:{fingerprint}", 900,
    json.dumps(rights))

signing_key = get_signing_key_from_ssm() # cached in Lambda memory
payload = { "sub": user_id, "tenant_id": tenant_id, "rights_fp": fingerprint,
    "channel": event.get("channel", "human"), "iat": int(time.time()),
    "exp": int(time.time()) + 900, "iss": "platform-jwt-svc",
    "aud": "agentcore-platform" }

return {"token": jwt.encode(payload, signing_key, algorithm="RS256")}

```

4.4 Rights Change Invalidation

When FT rights change in DynamoDB, a Stream triggers a Lambda that invalidates all ElastiCache keys for the affected user using Redis SCAN + DEL. The next request will cause a cache miss, falling back to DynamoDB (10–15 ms one-time penalty), which refreshes the cache with the updated rights.

4.5 Auth Matrix Across All Six Layers

Layer	Authn Mechanism	Authz Enforcement	Token Type
① Browser	Entra PKCE; HttpOnly cookie	None (pure UI)	Entra id_token
② CopilotRuntime	Verify Entra token (JWKS cache); no net call in hot path	Block on invalid/expired; inject JWT to all agent calls	Entra id_token → internal JWT
③ Human BFF	Token exchange → JWT Token Svc (VPC PrivateLink)	No rights in token — fingerprint only; VPC SG enforced	Internal RS256 JWT
④ API Gateway	RS256 JWT signature; aud/iss/exp validated	ElastiCache rights hydrate; route-scoped subset <5 KB	IAM policy + context
⑤ AgentCore Identity	OAuth 2.0 Bearer (Option A) or SigV4 (Option B MCP)	OIDC discovery URL config; allowed audiences + clients	Same internal JWT
⑥ Skill agents	Re-validate same JWT independently (defense-in-depth)	@requires_right per tool; tenant from JWT claims only	Same internal JWT

5 AgentCore Runtime + Strands Skills

GA Oct 2025 · Strands Agents 1.0 · ARM64 containers · 8-hr sessions

5.1 AgentCore GA Components (October 2025)

Amazon Bedrock AgentCore is now generally available in nine AWS Regions. The GA release adds A2A protocol support, identity-aware authorization, self-managed memory strategy, IAM authorization for MCP (in addition to OAuth), and OTEL-compatible observability with integration to Datadog, Dynatrace, Arize Phoenix, LangSmith, and Langfuse.

Component	GA Capabilities
AgentCore Runtime	Serverless microVM; 8-hr execution; complete session isolation; A2A server support; arm64 ECR containers; VPC-only network config; OAuth 2.0 + Entra ID integration
AgentCore Gateway	MCP server (tools/list, tools/call); Lambda/REST→MCP transformation; IAM + OAuth authorization; semantic tool search; connects to external MCP servers
AgentCore Memory	Session + long-term memory; AgentCoreMemorySessionManager for Strands; self-managed extraction strategy; CloudWatch metrics by default
AgentCore Identity	GetWorkloadAccessToken() for M2M; identity-aware authorization; refresh token vault; Entra/Okta/Cognito integration; OAuth scopes
AgentCore Observability	OTEL compatible; CloudWatch dashboards (latency, errors, token counts); spans and logs; trace visualisation; external provider integration
Built-in Tools	Code Interpreter (sandbox code execution); Browser Tool (managed web browser)

5.2 Strands Agents 1.0 — Skill Architecture

Strands Agents 1.0 (September 2025) provides four multi-agent primitives: agents-as-tools (hierarchical delegation), swarms (parallel execution), graph (explicit workflow DAG), and workflow (sequential pipelines). For this platform, the agents-as-tools pattern is used for skill invocation, with progressive disclosure via a Bedrock Knowledge Base skill registry.

Skill invocation pattern:

```
# orchestrator/agent.py (deployed to AgentCore Runtime)

from strands import Agent, tool
from strands.models import BedrockModel
from bedrock_agentcore import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

@tool
def retrieve_skill(query: str, tenant_id: str) -> str:
    """Semantic search Bedrock KB for the best skill matching the user's intent.
    Returns top-5 skills filtered to those the user has rights to use."""
    results = bedrock_kb.retrieve(
        knowledgeBaseId=SKILL_KB_ID,
        retrievalQuery={"text": query},
```

```

retrievalConfiguration={"vectorSearchConfiguration": {
"numberOfResults": 10,
"filter": {"equals": {"key": "tenant_id", "value": tenant_id}}
}}
)

user_rights = get_rights_from_context()

accessible = [r for r in results["retrievalResults"]
if skill_accessible(r["metadata"]["rights_req"], user_rights)][0:5]

return json.dumps([{"name": r["metadata"]["skill_name"],
"endpoint_arn": r["metadata"]["endpoint_arn"],
"description": r["content"]["text"]} for r in accessible])

@tool

def invoke_skill(skill_name: str, skill_arn: str, payload: dict) -> str:
    """Invoke a named skill agent. Forwards the user's JWT for independent rights check."""
    client = boto3.client("bedrock-agentcore-runtime")
    resp = client.invoke_agent_runtime(
        agentRuntimeArn=skill_arn,
        payload=json.dumps(payload),
        sessionId=get_current_session_id(),
        additionalModelRequestFields={
            "headers": {"Authorization": f"Bearer {get_current_user_token()}"}
        }
    )
    return resp["output"]

@app.entrypoint

def invoke(payload: dict, context) -> dict:
    tenant_id = context.requestContext.authorizer.tenant_id
    rights = json.loads(context.requestContext.authorizer.rights_json)
    session_mgr = AgentCoreMemorySessionManager(config, region_name='us-east-1')
    agent = Agent(
        model=BedrockModel(model_id="anthropic.claude-sonnet-4-5"),
        system_prompt=build_system_prompt(tenant_id, rights),
        tools=[retrieve_skill, invoke_skill],
        session_manager=session_mgr,
    )
    return {"response": str(agent(payload["message"]))}

```

5.3 Skill Agent Pattern — Rights Guard

```

# skills/finance_skill/agent.py

from functools import wraps

```

```

def requires_right(resource: str, action: str):
    """Decorator: validates FT rights BEFORE executing a tool. Defense-in-depth."""
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            rights = get_rights_from_context() # from JWT, not from payload
            if not has_right(rights, resource, action):
                raise PermissionError(f"Access denied: {resource}:{action}")
            validated_tenant = get_tenant_from_context() # from JWT claims only
            return fn(*args, **kwargs)
        return wrapper
    return decorator

@tool
@requires_right("finance:portfolio", "read")
def get_portfolio(account_id: str, tenant_id: str) -> dict:
    """Tenant-scoped portfolio lookup – tenant_id validated from JWT, not payload."""
    # DynamoDB with LeadingKey condition ensures tenant isolation at data layer
    ...

@tool
@requires_right("finance:calculations", "execute")
def run_valuation_calc(portfolio_id: str, model: str = "dcf") -> dict: ...

```

6 MCP + Tool Layer

AgentCore Gateway · IBM ContextForge · IBM API Connect · MCP Proxy for AWS

6.1 AgentCore Gateway (GA October 2025)

AgentCore Gateway acts as the tenant's internal tool catalog and MCP server. It transforms Lambda functions and REST APIs into MCP-compatible tools, provides semantic search across the tool catalog, and supports both IAM and OAuth 2.0 authorization for agent-to-tool interactions.

- **Tool registration:** Lambda ARNs, REST API OpenAPI specs, or existing MCP server URLs
- **Semantic search:** Agents query via natural language; Gateway returns top-K matching tools
- **Auth:** Inbound SigV4 (MCP Proxy for AWS) or OAuth 2.0 Bearer JWT
- **Cross-tenant:** Tenant B's Gateway can be invoked by Tenant A with scoped token via PrivateLink

6.2 IBM ContextForge — MCP Federation Gateway

IBM ContextForge (open source, runs on ECS Fargate in the platform VPC) acts as a federation hub for external APIs and A2A agents. It exposes a single MCP endpoint that aggregates tools from IBM API Connect, third-party MCP servers, and A2A agents. It handles JWT validation, per-tenant rate limiting (Redis-backed), and forwards X-Upstream-Auth credentials to downstream systems.

ContextForge configuration:

```
# docker-compose.contextforge.yml (ECS Task)

services:
  contextforge:
    image: ghcr.io/ibm/mcp-context-forge:latest
    environment:
      JWT_SECRET_KEY: "${JWT_PUBLIC_KEY}" # RS256 public key
      JWT_ALGORITHM: "RS256"
      JWKS_URL: "https://platform-jwt-svc.internal/jwks"
      DATABASE_URL: "${CONTEXTFORGE_DB_URL}" # RDS PostgreSQL
      REDIS_URL: "${ELASTICACHE_URL}" # Rate limiting
      TENANT_CLAIM_KEY: "tenant_id"
      ENFORCE_TENANT_ISOLATION: "true"
      MCPGATEWAY_UI_ENABLED: "false" # Disable UI in prod
```

6.3 IBM API Connect Integration

```
# Register IBM API Connect as an MCP server target in ContextForge

httpx.post(f"{CONTEXTFORGE_URL}/gateways", json={
  "name": "ibm-api-connect-prod",
  "url": "https://apic.internal.company.com/api/v2",
  "auth_type": "bearer",
  "auth_value": "${IBM_API_CONNECT_TOKEN}", # from SSM at runtime
  "openapi_spec_url": "https://apic.internal.company.com/openapi.json",
  "tenant_id": "platform",
  "rate_limit_per_minute": 500,
  "tags": ["ibm", "enterprise", "external"]
})
```

```
} )
```

6.4 Internet Call Minimization

A design invariant is that zero calls transit the public internet during normal operation. The following VPC endpoint pattern achieves this:

- bedrock-runtime, bedrock-agent-runtime, bedrock-agentcore
- ssm, secretsmanager (eliminates internet calls for secrets)
- dynamodb, s3, elasticache (all internal DDB / cache / session calls)
- execute-api (APIGW invocation stays within AWS backbone)
- IBM ContextForge deployed in same VPC — no internet for external tools
- Cross-tenant AgentCore calls via PrivateLink — not over internet

7 Cross-Tenant A2A Design

A2A protocol · MCP Proxy for AWS · PrivateLink · Token propagation

7.1 Cross-Tenant Token Propagation

Step	What happens	Token used
1	Orchestrator A decides to call a skill in Tenant B	User JWT (A)
2	AgentCore Identity: GetWorkloadAccessToken(scope=tenantB:read, audience=tenantB-agentcore)	Workload token (scoped)
3	MCP Proxy for AWS signs request with SigV4; sends via PrivateLink to Tenant B AgentCore	SigV4 + scoped JWT
4	AgentCore Identity (B) validates: SigV4 OK; scoped JWT OK; cross-tenant policy allows caller A	Scoped JWT validated
5	MCP server B processes tool call with propagated context headers	Context headers
6	Result streamed back via SSE; OTEL span recorded in both tenant A and B CloudWatch	None (response)

7.2 Cross-Tenant Policy Table (DynamoDB)

```
# Cross-tenant access policy record
{
  "pk": "caller_tenant=BU1",
  "sk": "target_tenant=BU2",
  "allowed_scopes": ["read", "invoke:search"],
  "allowed_agent_arns": ["arn:aws:bedrock-agentcore:...:runtime/BU1/orchestrator"],
  "expiry": "2026-12-31",
  "approved_by": "platform-admin",
  "audit_trail": true
}
```

The policy table is managed by the platform team and acts as a whitelist. AgentCore Identity checks this table before issuing cross-tenant workload tokens. Expired or absent policies result in 403 — no escalation path.

8 Observability + Operations

AgentCore Observability · CloudWatch · OTEL · Audit trail

8.1 AgentCore Observability (GA October 2025)

AgentCore Observability is OTEL-compatible and integrates natively with CloudWatch, Datadog, Dynatrace, Arize Phoenix, LangSmith, and Langfuse. It provides end-to-end trace visualisation across Runtime, Gateway, Memory, Identity, and Built-in Tools.

Signal type	What is captured	Destination
Metrics	Invocations, latency (p50/p99), token counts, error rates, throttling, session counts	CloudWatch (default, always on)
Spans / Traces	End-to-end agent execution; tool call durations; cross-tenant A2A hops; memory access	CloudWatch X-Ray / OTEL collector
Logs	Application logs from agent containers; auth failures; rights violations	CloudWatch Logs (S3 / Firehose optional)
Audit trail	Every tool call: user_id, tenant_id, tool_name, arguments hash, outcome, latency	DynamoDB audit table (immutable, 7yr retention)
Evaluation	13 built-in evaluators + custom evaluators; trajectory inspection; quality scoring	CloudWatch Observability dashboard

9 CDK Infrastructure Patterns

TypeScript CDK · Per-tenant stacks · VPC endpoints · ECR

```
// lib/agentcore-tenant-stack.ts (key excerpts)

export class AgentCoreTenantStack extends cdk.Stack {
  constructor(scope, id, props: TenantProps) {
    const { tenantId } = props;

    // One ECR repo per agent type – immutable tags, scan on push
    const orchestratorRepo = new ecr.Repository(this, 'OrchestratorRepo', {
      repositoryName: `${tenantId}/orchestrator`,
      imageTagMutability: ecr.TagMutability.IMMUTABLE,
      imageScanOnPush: true,
    });

    // AgentCore execution role – scoped to this tenant only
    const agentRole = new iam.Role(this, 'AgentRole', {
      assumedBy: new iam.ServicePrincipal('bedrock-agentcore.amazonaws.com'),
      inlinePolicies: { agentPolicy: new iam.PolicyDocument({ statements: [
        new iam.PolicyStatement({ // Bedrock model invocation
          actions: ['bedrock:InvokeModel', 'bedrock:InvokeModelWithResponseStream'],
          resources: ['arn:aws:bedrock:*::foundation-model/*'],
        }),
        new iam.PolicyStatement({ // AgentCore – tenant-scoped only
          actions: ['bedrock-agentcore:InvokeAgentRuntime',
            'bedrock-agentcore:GetWorkloadAccessToken'],
          resources: [`arn:aws:bedrock-agentcore:*::agent-runtime/${tenantId}/*`],
          conditions: { StringEquals: { 'aws:ResourceTag/tenantId': tenantId } },
        }),
        new iam.PolicyStatement({ // SSM – tenant path only
          actions: ['ssm:GetParameter', 'ssm:GetParametersByPath'],
          resources: [`arn:aws:ssm:*::parameter/tenants/${tenantId}/*`],
        }),
        new iam.PolicyStatement({ // DynamoDB – LeadingKey tenant isolation
          actions: ['dynamodb:GetItem', 'dynamodb:Query', 'dynamodb:PutItem'],
          resources: ['*'],
          conditions: { StringEquals: { 'dynamodb:LeadingKeys': [tenantId] } },
        }),
      ]}),
    });

    // VPC endpoints – ALL AWS calls stay within AWS backbone (zero internet)
```

```
[ 'bedrock-runtime', 'bedrock-agent-runtime', 'bedrock-agentcore',  
'ssm', 'secretsmanager', 'dynamodb', 's3', 'elasticache' ].forEach(svc =>  
vpc.addInterfaceEndpoint(`${svc}Ep`, {  
  service: new ec2.InterfaceVpcEndpointService(  
    com.amazonaws.${this.region}.${svc}`)  
  })  
});  
}  
}
```

10 Anti-Patterns & Best Practices

Design decisions that matter in production

Anti-pattern	Why it fails	Correct pattern
Full FT rights set in JWT	Lambda Authorizer 8 KB context limit. 200 rights tuples $\approx$ 12 KB. Silent truncation or 502.	SHA-256 fingerprint in token. Hydrate from ElastiCache in authorizer. <2 ms overhead.
Shared AgentCore Runtime across tenants	AWS docs state the MCP server deployment is NOT multi-tenant safe. Session isolation only.	One Runtime deployment per BU tenant. Ideally separate AWS accounts. Tag all resources.
All skill descriptions to every LLM call	200 skills $\times$ 200 tokens = 40K tokens in system prompt. Context exhaustion + cost explosion.	Strands retrieve-tool: semantic search Bedrock KB, inject top-5 per query only.
Trust orchestrator's rights claims in skills	Prompt injection or compromised orchestrator can fabricate rights in tool call payload.	Every skill validates the user JWT independently. @requires_right reads JWT context.
Static API keys in agent containers	Keys in ECR image layers exposed in scan results and container introspection.	AgentCore Identity GetWorkloadAccessToken(). Zero static creds in code. 15-min tokens.
Cross-tenant calls over public internet	Egress cost, 50–200 ms latency, expanded attack surface.	VPC-only network config on AgentCore + PrivateLink VPC endpoints.
One BFF for human and machine callers	Human BFF: WebSocket + PKCE + session. Machine BFF: stateless + M2M + async. Incompatible.	Separate ECS services per channel. Same API Gateway + Authorizer downstream.
Entra token passed directly to AgentCore	Entra token audience doesn't match AgentCore's OIDC config. Hard to add FT rights.	Custom JWT Service exchanges token. Adds FT fingerprint. Controls audience/issuer.
tenant_id from request payload (not JWT)	Client can forge any tenant_id in the request body — complete tenant bypass.	tenant_id ALWAYS extracted from JWT claims in authorizer context. Never from body.
Storing Entra token in React state/localStorage	XSS attack steals token. No rotation. No server-side invalidation possible.	HttpOnly session cookie set by BFF during PKCE callback. Never in JS-accessible storage.
No toolFilter in MCPAppsMiddleware (Option B)	UI shows all tools including ones the user can't use — UX confusion + info leakage.	toolFilter: check required_right annotation against user rights before exposing.

11 Implementation Roadmap

Phased delivery · Critical path · Risk-ordered

Phase	What to build	Key risks	Est. duration
P0 — Foundation	Entra PKCE + Client Creds flows. Custom JWT Token Service (Lambda + SSM). ElastiCache cluster. Lambda Authorizer with rights fingerprint. DynamoDB FT rights table.	Rights size budget; test with 200-tuple rights set before commit	2 weeks
P1 — First Agent	CDK stack for Tenant A. Strands orchestrator on AgentCore Runtime. One skill agent. AgentCore Memory (AgentCoreMemorySessionManager).	AgentCore CDK L1 constructs are new; validate VPC-only routing	2 weeks
P2 — CopilotKit UI	Next.js BFF + CopilotRuntime. Option A HttpAgent wired to AgentCore. onBeforeRequest JWT injection. CopilotSidebar + useCoAgent. Human-in-loop ApprovalCard component.	AG-UI version pinning; test threadId ↔ sessionId mapping	1.5 weeks
P3 — Skill Registry	Bedrock Knowledge Base for skill documents. retrieve-tool in Strands orchestrator. 3–5 production skills with @requires_right guards.	KB embedding pipeline; rights_req metadata in skill documents	1.5 weeks
P4 — Tool Layer	AgentCore Gateway for Tenant A. IBM ContextForge on ECS Fargate. First IBM API Connect API wrapped as MCP tool. Option B MCPAppsMiddleware.	ContextForge JWT config; IBM API Connect OpenAPI completeness	1.5 weeks
P5 — Second Tenant	Replicate CDK stack for Tenant B. Cross-tenant policy DynamoDB table. A2A invocation from Tenant A orchestrator to Tenant B. PrivateLink between accounts.	Cross-account PrivateLink; IAM cross-account trust configuration	2 weeks
P6 — Machine Channel	Machine BFF (Lambda). SQS async invocation. POST /invoke → 202 → GET /results. Client credentials flow for C/D users.	SQS visibility timeout vs AgentCore 8-hr window; async result storage	1 week
P7 — Observability	OTEL traces end-to-end. AgentCore Observability dashboards. CloudWatch alarms for error rate + latency. Audit DynamoDB table with 7yr TTL. Datadog / Dynatrace integration (optional).	Trace correlation across tenant boundaries; PII in log scrubbing	1 week
P8 — Hardening	Penetration test on JWT service. Rights invalidation load test. Chaos testing for session recovery. Lambda Authorizer cache-miss p99 baseline. Runbook for rights cache flush.	Cache miss spike under rights churn; cold-start Lambda Authorizer latency	1 week

Critical path note:

P0 (auth foundation) is the single most important phase to get right before any other work. Every downstream layer depends on the rights fingerprint pattern working correctly at scale. Before committing P0 to production, test with your largest expected rights set (simulate 200 resource tuples) and validate that the Lambda Authorizer context stays under 5 KB on the worst-case route. The 8 KB hard limit must never be approached — route-scoped subsetting in the authorizer is not optional.

Document Information

Prepared by

Platform Architecture Team

Research sources	AWS docs (AgentCore GA Oct 2025), Strands 1.0 (Sep 2025), CopilotKit 1.51 (Jan 2026), IBM ContextForge (2025),
Next review	Q3 2026
Confidentiality	Internal use only — do not distribute externally